

Biglietteria del concerto

Esercizi di multithreading in Python — Advanced Computer Programming

Cinque esercizi indipendenti, in difficoltà crescente, incentrati esclusivamente su `thread` e sincronizzazione. Non è richiesto alcun costruito avanzato di programmazione a oggetti oltre a una classe di stato elementare, e non vanno usati né `queue.Queue` né meccanismi di pipe: tutta la sincronizzazione va costruita con le primitive di `threading`.

Contesto comune

Un botteghino mette in vendita un numero fisso di biglietti e più sportelli, rappresentati da `thread` distinti, vendono contemporaneamente. L'unico stato condiviso è il seguente:

```
class Botteghino:
    def __init__(self, totale):
        self.disponibili = totale
        self.venduti = 0
        self.log = []           # elenco delle vendite effettuate
```

In tutti gli esercizi si assumono otto `thread`, cinquecento biglietti complessivi e cento tentativi di vendita per ciascun `thread`.

Esercizio 1 — Provocare la race condition

Scrivi una funzione `vendi(botteghino, id_sportello)` che, se `disponibili` è maggiore di zero, decrementa `disponibili`, incrementa `venduti` e aggiunge la tupla `(id_sportello, numero_biglietto)` alla lista `log`. La funzione va scritta **senza alcun meccanismo di lock**.

Fra la lettura di `disponibili` e la sua riscrittura inserisci una chiamata a `time.sleep(0)`: forza il rilascio del GIL e rende l'errore deterministico anziché sporadico.

Verifica. Al termine dell'esecuzione `venduti` deve risultare diverso da `len(log)`, oppure `disponibili` deve assumere un valore negativo. Se l'anomalia non si manifesta, aumenta il numero di `thread`. Annota il valore finale di `disponibili` per confrontarlo con l'esercizio successivo.

Esercizio 2 — Correzione con Lock

Riscrivi la stessa funzione introducendo un `threading.Lock`, impiegato come context manager attraverso l'istruzione `with`.

Il risultato deve essere esattamente: `disponibili == 0`, `venduti == 500`, `len(log) == 500` e nessun numero di biglietto duplicato, condizione che puoi verificare confrontando la lunghezza dell'insieme dei numeri emessi con cinquecento.

Domanda. La sezione critica deve comprendere anche l'operazione di `append` sulla lista `log`? Motiva la risposta esaminando che cosa accade se la escludi dal blocco protetto.

Esercizio 3 — Semaforo e terminali di stampa

Gli sportelli aperti sono otto, ma i terminali di stampa disponibili sono soltanto tre. Aggiungi dopo la vendita una fase `stampa_biglietto()` della durata di cinque centesimi di secondo, che possa essere eseguita da un massimo di tre `thread` contemporaneamente.

Utilizza un `threading.Semaphore(3)`. Per dimostrare il rispetto del vincolo mantieni un contatore `attivi`, protetto da un lock separato, e registra il valore massimo raggiunto: non deve mai superare tre.

Domanda. Perché non è possibile riutilizzare qui il `Lock` dell'esercizio precedente al posto del semaforo?

Esercizio 4 — Buffer condiviso con Condition

L'architettura si ribalta in uno schema produttore/consumatore, ma il buffer va costruito interamente a mano.

```
class BufferRichieste:
    def __init__(self, capienza):
        self.capienza = capienza
        self.elementi = []           # lista usata come coda FIFO
        self.condizione = threading.Condition()
```

Implementa due metodi. Il metodo `inserisci(richiesta)` fa attendere il produttore quando il buffer è pieno, altrimenti accoda l'elemento e risveglia i `thread` in attesa. Il metodo `preleva()` fa attendere il consumatore quando il buffer è vuoto, altrimenti estrae il primo elemento e risveglia i `thread` in attesa.

Un `thread` produttore genera cinquecento richieste con capienza venti, mentre quattro `thread` consumatori le evadono invocando la versione corretta di `vendi()` prodotta nell'esercizio 2.

Vincoli da rispettare

- Ogni chiamata a `wait()` deve trovarsi all'interno di un ciclo `while` e mai dentro un `if`. Scrivi dapprima la versione con `if`, poi passa al `while` e annota in un commento che cosa cambia, considerando sia i risvegli spuri sia la possibilità che il predicato venga invalidato da un altro `thread` prima che il lock venga riacquisito.
- Usa `notify_all()` e non `notify()`. Prova successivamente a sostituirlo e osserva se il programma si blocca: quando produttori e

consumatori attendono sulla medesima condizione, il risveglio può essere assegnato al thread sbagliato.

3. L'oggetto `Condition` incorpora già un lock: impiegalo come context manager con `with self.condizione:` senza aggiungere un `Lock` separato.

Terminazione

In assenza di una coda di libreria la sentinella va gestita esplicitamente: il produttore inserisce in fondo quattro valori `None`, uno per ciascun consumatore, e ogni consumatore termina quando ne preleva uno. Presta attenzione al fatto che un consumatore il quale preleva la sentinella e termina senza risvegliare gli altri può lasciare i restanti bloccati in attesa: assicurati che l'uscita sia sempre preceduta da un `notify_all()`.

Verifica. Al termine devono valere `venduti == 500` e `len(buffer.elementi) == 0`, e il programma deve uscire autonomamente. Applica un `join()` su tutti i thread e imponi un timeout complessivo: se l'esecuzione supera i dieci secondi sei in presenza di un deadlock.

Esercizio 5 — Chiusura anticipata con Event

Il botteghino chiude dopo due secondi anche se restano biglietti invenduti, e il main segnala la chiusura impostando un `threading.Event`.

Il punto delicato è il risveglio. I consumatori possono infatti trovarsi fermi in `wait()` su un buffer vuoto, e un `Event` impostato **non li risveglia**, perché si tratta di due meccanismi di sincronizzazione distinti e indipendenti.

- I consumatori devono controllare `evento_chiusura.is_set()` all'interno del predicato del `while`, non soltanto all'inizio del ciclo esterno.
- Il main, immediatamente dopo `evento_chiusura.set()`, deve eseguire `with buffer.condizione: buffer.condizione.notify_all()` per sbloccare chi è in attesa.
- L'attesa dei due secondi nel main va realizzata con `evento_chiusura.wait(timeout=2)` e non con `time.sleep(2)`.

Verifica. Il totale venduto deve risultare inferiore a cinquecento, la somma `venduti + disponibili` deve restare pari a cinquecento in ogni istante e nessun thread deve sopravvivere al main, condizione controllabile con `threading.enumerate()` dopo i `join()`.

Domanda finale. Perché in questo schema il `notify_all()` eseguito dal main deve trovarsi all'interno del blocco `with` sulla condizione, e che cosa accadrebbe invocandolo senza possedere il lock?

Prospetto delle primitive

Esercizio	Primitiva	Concetto verificato
1	nessuna	Non atomicità delle operazioni composte e ruolo del GIL
2	<code>Lock</code>	Mutua esclusione e ampiezza della sezione critica
3	<code>Semaphore</code>	Accesso concorrente limitato a n risorse
4	<code>Condition</code>	Attesa su predicato, risvegli spuri, terminazione con sentinella
5	<code>Event</code>	Segnalazione fra thread e interazione con l'attesa su condizione

Ordine consigliato. Esegui 1 e 2 di seguito sullo stesso codice, così da isolare l'effetto del lock. Gli esercizi 3, 4 e 5 presuppongono la versione corretta prodotta nel secondo.